## My first three checks

1. Reproduce the problem and establish the scope.
   - ✓ Open the article in a private browser window and a normal session.
   - ✓ Compare the URL, timestamp, article revision, and visible content.
   - ✓ Check whether the discrepancy affects all users, a particular region, signed-in users, or a particular device.

2. Inspect the caching layers.
   - ✓ Check CDN and reverse-proxy cache status and response headers.
   - ✓ Compare the response from the public URL with the origin response, where safely possible.
   - ✓ Inspect WordPress page-cache, object-cache, and browser-cache behaviour.
   - ✓ Check whether cache invalidation ran after the release.

3. Verify the code and content actually deployed.
   - ✓ Confirm the release identifier and deployment status.
   - ✓ Inspect the WordPress article revision and publication state.
   - ✓ Check application logs, failed deployment steps, and any recent changes to the theme or publishing workflow.
   - ✓ Confirm that the intended correction exists in the origin response and database, rather than assuming the problem is a cache.

## How I would distinguish the causes

| Evidence | Likely direction of investigation |
|---|---|
| Origin contains the correction but a cached response does not | Caching or invalidation issue |
| Both origin and cache return old content | Publication, content, or deployment issue |
| HTML is current but the browser displays old content | Client-side caching, JavaScript, or rendering issue |

| | |
|---|---|
| Only signed-in or anonymous users see stale content | Different cache rules, personalization, or authentication-specific behaviour |
| Only some servers return old content | Deployment inconsistency or an upstream cache |

These are diagnostic clues, not proof on their own.

## When I would roll back

I would roll back if there is evidence that the release introduced a material regression, such as broken article rendering, data corruption, a security issue, or an unreliable publishing path. I would not automatically roll back a valid content correction merely because a cache is stale.

If the application is healthy and the origin has the correct article, I would first address the relevant cache invalidation or TTL issue. If rollback is required, I would use the established deployment procedure and ensure that the rollback itself does not overwrite the corrected article content.

## How I would verify recovery

- ✓ Request the article repeatedly from affected and unaffected sessions.
- ✓ Confirm the article content and release version are correct.
- ✓ Verify CDN and origin responses agree after the appropriate cache refresh.
- ✓ Check application errors and editorial publishing functionality.
- ✓ Monitor the affected URL after recovery.

## Two-sentence update to the editor

I'm checking whether the inconsistent article versions are coming from a cache that has not refreshed or from the release or publishing process itself, and I'll avoid another release until we identify the cause. I'll confirm when the corrected article is consistent across the affected reader sessions and let you know whether a targeted cache fix or a rollback was necessary.